Volume 03 - Book 03.07 Runtime Ecosystem

Version v9 draft

README.md

Book 03.07 - Runtime Ecosystem

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-README

Purpose

Define the Runtime Ecosystem blueprint package and its subsystem decomposition as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for the Runtime Ecosystem blueprint package and its subsystem decomposition. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for the Runtime Ecosystem blueprint package and its subsystem decomposition.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for the Runtime Ecosystem blueprint package and its subsystem decomposition.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.

- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.
- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeEcosystemDefined
- RuntimeEcosystemRegistered
- RuntimeEcosystemStateChanged
- RuntimeEcosystemReviewRequested
- RuntimeEcosystemGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.

- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.
- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Ecosystem has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

INDEX.md

Book 03.07 - Runtime Ecosystem

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-INDEX

Purpose

Define the Runtime Ecosystem blueprint package and its subsystem decomposition as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for the Runtime Ecosystem blueprint package and its subsystem decomposition. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for the Runtime Ecosystem blueprint package and its subsystem decomposition.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for the Runtime Ecosystem blueprint package and its subsystem decomposition.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.
- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeEcosystemDefined
- RuntimeEcosystemRegistered
- RuntimeEcosystemStateChanged
- RuntimeEcosystemReviewRequested
- RuntimeEcosystemGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.

- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.
- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Ecosystem has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-01-runtime-ecosystem-overview.md

Book 03.07.01 - Runtime Ecosystem Overview

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-001

Purpose

Define Runtime Ecosystem Overview within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Ecosystem Overview within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Ecosystem Overview within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Ecosystem Overview within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.
- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeEcosystemOverviewDefined
- RuntimeEcosystemOverviewRegistered
- RuntimeEcosystemOverviewStateChanged
- RuntimeEcosystemOverviewReviewRequested
- RuntimeEcosystemOverviewGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.

- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.
- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Ecosystem Overview has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-02-runtime-kernel.md

Book 03.07.02 - Runtime Kernel

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-002

Purpose

Define Runtime Kernel within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Kernel within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Kernel within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Kernel within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeKernelDefined
- RuntimeKernelRegistered
- RuntimeKernelStateChanged
- RuntimeKernelReviewRequested
- RuntimeKernelGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Kernel has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-03-model-runtime.md

Book 03.07.03 - Model Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-003

Purpose

Define Model Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Model Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Model Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Model Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ModelRuntimeDefined
- ModelRuntimeRegistered
- ModelRuntimeStateChanged
- ModelRuntimeReviewRequested
- ModelRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Model Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

Book 03.07.04 - Plugin Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-004

Purpose

Define Plugin Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Plugin Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Plugin Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Plugin Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- PluginRuntimeDefined
- PluginRuntimeRegistered
- PluginRuntimeStateChanged
- PluginRuntimeReviewRequested
- PluginRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Plugin Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-05-connector-runtime.md

Book 03.07.05 - Connector Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-005

Purpose

Define Connector Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Connector Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Connector Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Connector Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ConnectorRuntimeDefined
- ConnectorRuntimeRegistered
- ConnectorRuntimeStateChanged
- ConnectorRuntimeReviewRequested
- ConnectorRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Connector Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-06-provider-runtime.md

Book 03.07.06 - Provider Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-006

Purpose

Define Provider Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Provider Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Provider Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Provider Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ProviderRuntimeDefined
- ProviderRuntimeRegistered
- ProviderRuntimeStateChanged
- ProviderRuntimeReviewRequested
- ProviderRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Provider Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-07-workflow-runtime.md

Book 03.07.07 - Workflow Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-007

Purpose

Define Workflow Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Workflow Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Workflow Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Workflow Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- WorkflowRuntimeDefined
- WorkflowRuntimeRegistered
- WorkflowRuntimeStateChanged
- WorkflowRuntimeReviewRequested
- WorkflowRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Workflow Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-08-mission-runtime.md

Book 03.07.08 - Mission Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-008

Purpose

Define Mission Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Mission Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Mission Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Mission Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- MissionRuntimeDefined
- MissionRuntimeRegistered
- MissionRuntimeStateChanged
- MissionRuntimeReviewRequested
- MissionRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Mission Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-09-service-runtime.md

Book 03.07.09 - Service Runtime

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-009

Purpose

Define Service Runtime within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Service Runtime within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Service Runtime within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Service Runtime within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- ServiceRuntimeDefined
- ServiceRuntimeRegistered
- ServiceRuntimeStateChanged
- ServiceRuntimeReviewRequested
- ServiceRuntimeGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Service Runtime has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

Book 03.07.10 - Runtime Scheduler

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-010

Purpose

Define Runtime Scheduler within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Scheduler within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Scheduler within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Scheduler within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeSchedulerDefined
- RuntimeSchedulerRegistered
- RuntimeSchedulerStateChanged
- RuntimeSchedulerReviewRequested
- RuntimeSchedulerGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Scheduler has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-11-runtime-isolation.md

Book 03.07.11 - Runtime Isolation

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-011

Purpose

Define Runtime Isolation within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Isolation within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Isolation within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Isolation within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeIsolationDefined
- RuntimeIsolationRegistered
- RuntimeIsolationStateChanged
- RuntimeIsolationReviewRequested
- RuntimeIsolationGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Isolation has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-12-runtime-policy.md

Book 03.07.12 - Runtime Policy

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-012

Purpose

Define Runtime Policy within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Policy within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Policy within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Policy within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimePolicyDefined
- RuntimePolicyRegistered
- RuntimePolicyStateChanged
- RuntimePolicyReviewRequested
- RuntimePolicyGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Policy has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-13-runtime-observability.md

Book 03.07.13 - Runtime Observability

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-013

Purpose

Define Runtime Observability within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Observability within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Observability within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Observability within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeObservabilityDefined
- RuntimeObservabilityRegistered
- RuntimeObservabilityStateChanged
- RuntimeObservabilityReviewRequested
- RuntimeObservabilityGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Observability has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-14-runtime-recovery.md

Book 03.07.14 - Runtime Recovery

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-014

Purpose

Define Runtime Recovery within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Recovery within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Recovery within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Recovery within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeRecoveryDefined
- RuntimeRecoveryRegistered
- RuntimeRecoveryStateChanged
- RuntimeRecoveryReviewRequested
- RuntimeRecoveryGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Recovery has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-15-runtime-scaling.md

Book 03.07.15 - Runtime Scaling

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-015

Purpose

Define Runtime Scaling within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Scaling within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Scaling within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Scaling within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeScalingDefined
- RuntimeScalingRegistered
- RuntimeScalingStateChanged
- RuntimeScalingReviewRequested
- RuntimeScalingGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Scaling has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

Book 03.07.16 - Runtime Events

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-016

Purpose

Define Runtime Events within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Events within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Events within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Events within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeEventsDefined
- RuntimeEventsRegistered
- RuntimeEventsStateChanged
- RuntimeEventsReviewRequested
- RuntimeEventsGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Events has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

Book 03.07.17 - Runtime Storage

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-017

Purpose

Define Runtime Storage within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Storage within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Storage within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Storage within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeStorageDefined
- RuntimeStorageRegistered
- RuntimeStorageStateChanged
- RuntimeStorageReviewRequested
- RuntimeStorageGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Storage has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-18-runtime-security.md

Book 03.07.18 - Runtime Security

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-018

Purpose

Define Runtime Security within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Security within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Security within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Security within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeSecurityDefined
- RuntimeSecurityRegistered
- RuntimeSecurityStateChanged
- RuntimeSecurityReviewRequested
- RuntimeSecurityGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Security has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

book-03-07-19-runtime-testing.md

Book 03.07.19 - Runtime Testing

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-019

Purpose

Define Runtime Testing within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Testing within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Testing within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Testing within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeTestingDefined
- RuntimeTestingRegistered
- RuntimeTestingStateChanged
- RuntimeTestingReviewRequested
- RuntimeTestingGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Testing has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.

Book 03.07.20 - Runtime Roadmap

Version: v9 draft

Status: Draft

Document ID: MAL-V03-B03-07-020

Purpose

Define Runtime Roadmap within the Runtime Ecosystem blueprint as part of Volume 03 Master Blueprint for Mariam AI Enterprise OS.

Scope

This document covers architecture-level responsibilities, contracts, interfaces, events, storage expectations, security controls, metrics, testing, acceptance criteria, and traceability for Runtime Roadmap within the Runtime Ecosystem blueprint. It does not implement product code or bypass the documentation-first governance model.

Responsibilities

- Establish a canonical blueprint boundary for Runtime Roadmap within the Runtime Ecosystem blueprint.
- Convert strategic roadmap intent into implementation-ready subsystem contracts.
- Coordinate with Enterprise Core identity, runtime, event, storage, security, and observability services.
- Preserve governance, traceability, reviewability, and rollback expectations for high-impact behavior.
- Provide enough detail for future specifications, testing guides, operations guides, and implementation issues.

Inputs

- Volume 00 Enterprise Constitution principles for governance, security, quality, engineering, AI, DNA, swarm, and evolution.
- Volume 01 Master Vision strategy for enterprise outcomes, knowledge leverage, marketplace expansion, and autonomous work.
- Volume 02 Master Roadmap phase guidance corresponding to Runtime Ecosystem.
- Earlier Volume 03 blueprint books for Enterprise Core, Enterprise Organization, AI Society, Knowledge System, and Capability System where dependencies apply.
- Domain requirements, user intent, policy context, runtime constraints, and release acceptance evidence.

Outputs

- Blueprint contract and subsystem boundaries for Runtime Roadmap within the Runtime Ecosystem blueprint.
- Canonical records, events, metrics, storage expectations, and security controls.
- Traceable inputs for Volume 06 specifications and later specialist volumes.
- Acceptance evidence that downstream implementation can use before writing system code.

Interfaces

- Enterprise Core kernel, runtime manager, event bus, object manager, identity access, storage, security, and observability interfaces.

- Enterprise Organization decision, approval, escalation, KPI, and operating rhythm interfaces.
- AI Society agent, planning, execution, review, validation, and governance interfaces.
- Knowledge System graph, store, search, indexing, provenance, and quality interfaces.
- Capability System registry, matching, lifecycle, benchmark, governance, and execution interfaces.

Events

- RuntimeRoadmapDefined
- RuntimeRoadmapRegistered
- RuntimeRoadmapStateChanged
- RuntimeRoadmapReviewRequested
- RuntimeRoadmapGovernanceBlocked

Storage

- Canonical records for definitions, versions, owners, state, policy classification, and dependency metadata.
- Operational journals for lifecycle changes, execution evidence, review outcomes, exceptions, and acceptance decisions.
- Audit metadata linking user intent, agent or service identity, source context, runtime, model or provider choice, and policy status.
- Retention metadata that separates draft, active, archived, deprecated, and superseded records.

Security

- Apply least privilege to users, agents, services, providers, plugins, connectors, runtimes, and storage records.
- Deny protected actions when required permissions, policy checks, evidence, or human review gates are missing.
- Classify inputs, outputs, events, logs, metrics, memory, and exported artifacts before sharing or persistence.
- Audit privileged actions, denied actions, governance exceptions, configuration changes, and lifecycle transitions.
- Prevent generated or inferred outputs from becoming authoritative without explicit acceptance rules.

Metrics

- Adoption rate, successful completion rate, rejection rate, rework count, and user trust indicators.
- Latency, cost, throughput, availability, error rate, fallback rate, and recovery time.
- Quality score, benchmark pass rate, validation pass rate, review backlog, and stale-record count.
- Security denial count, governance exception count, policy violation count, and audit completeness.

Testing

- Contract tests for required inputs, outputs, interfaces, event envelopes, storage records, and lifecycle states.
- Security tests for tenant isolation, least privilege, protected action denial, redaction, and audit integrity.
- Quality tests for correctness, repeatability, ranking or routing behavior, recovery paths, and acceptance evidence.
- Integration tests across Enterprise Core, Organization, AI Society, Knowledge System, and Capability System dependencies.

- Release tests verifying Markdown inclusion, PDF generation, ZIP packaging, manifest checksums, and release notes.

Acceptance Criteria

- Runtime Roadmap has explicit ownership, lifecycle, interfaces, events, storage, security, metrics, and testing expectations.
- The blueprint is detailed enough for downstream specifications without requiring implementation code in this repository.
- Governance, security, and review controls are visible and traceable.
- Release artifacts include this Markdown file, the book PDF, MANIFEST.json, release notes, and the versioned ZIP.

Traceability

- MAL-V00-B009
- MAL-V00-B010
- MAL-V00-B011
- MAL-V00-B012
- MAL-V02-B007
- MAL-V03-B03-01-004
- MAL-V03-B03-05-003

Version History

Version	Date	Status	Notes
v9 draft	2026-06-29	Draft	Initial Runtime Ecosystem blueprint content for Mariam Architecture Library v9.